Smalltalk y otros metamodelos

En donde vamos a ver el metamodelo de otros lenguajes, y la diferencia entre “lenguaje” y “sistema”.

Universidad Nacional de Quilmes

Objetivos de la clase de hoy

- Conocer la estructura principal de los metamodelos de otros lenguajes de programación (en particular, Smalltalk y Java).
- Ver algunos temas relacionados a la historia de la programación orientada a objetos.
- Explorar la diferencia entre dos maneras de ver a la programación: centrada en el lenguaje, y centrada en el sistema.
    - Para los últimos dos puntos, nos vamos a concentrar en Smalltalk.

Vanessa Freudenberg

Smalltalk-72

```
to box var / x y size tilt (

   ◊draw ⇒ (☺ place x y turn tilt. square size.)
   ◊undraw ⇒ (☺ white. SELF draw. ☺ black)
   ◊redraw ⇒ (SELF undraw. :. SELF draw.)
   ◊turn ⇒ (SELF redraw ☞tilt ← tilt + :.)
   ◊grow ⇒ (SELF redraw ☞size ← size + :.)
   ◊move ⇒ (SELF redraw (☞x ← :. ☞y ← :.))
   ◊'s  ⇒ (☞var ← §. ◊← ⇒ (⇑var ← :.) ⇑ var eval)
   ◊is  ⇒ (◊box ⇒ (⇑☞box) ◊? ⇒ (⇑☞box) §. ⇑false)

   isnew ⇒ (
      ☞x ← ☞y ← 256.
      ☞size ← 50.
      ☞tilt ← 0.
      SELF draw
   )

)!
```

Bootstrapping (ALLDEFS)

'This file is an annotated version of sysdefs.
Comments (by Dan Ingalls and Diana Merry) are in string-quotes like this.
The code portion of this file is copyright Xerox Corp. *1974*'

'Phase I.
The bootstrap process sets up a global dictionary. It then reads input lines,
looking specifically for the defining word, "to", and calling its code directly.'

to to x (CODE *19*)
'Now definitions can be made by evaluating "to" in ST code'

```
to : (CODE 18)
      'to : name

      ↓☞ ⇒(:☞name nil ⇒(⇑name←caller message quotefetch)
                        (⇑caller message quotefetch))

      Fetch the next thing in the message stream unevaluated
      and bind it to the name if one is there.

      ↓# ⇒(:☞name nil ⇒(⇑name←caller message referencefetch)
                        (⇑caller message referencefetch))

      Fetch the reference to next thing in the message stream
      and bind it to the name if one is there.

            (:☞name nil ⇒(⇑name← caller message evalfetch)
                  ⇑caller message evalfetch)

      Fetch the next thing in the message stream evaluated
      and bind it to the name if one is there.'
```

to ↓ (CODE *17*)

':☞token. token=caller.message.code[caller.message.pc]⇒
(caller.message.pc←caller.message.pc+*1*. ⇑true) ⇑false.
That is, if a match for the token is found in the message, then
gobble it up and return true, else return false.'

to ⇓ (CODE *36*)

'Fetch the next token quoted -- equivalent to (:☞).'

to ⇑ (CODE *13*)

':x. then do a return, and apply x to any further message. Note
that in (... ⇑x+*3*. ☞y←y-*2*), the assignment to y will never
happen, since ⇑ causes a return.'

to ☞ (CODE *9*)

'⇑:☞. That is, get the next thing in the message stream unevalled
and active return it (which causes it to be applied to the message).'

to # (:#)

'Returns a REFERENCE to its arguments binding.'

Smalltalk-78

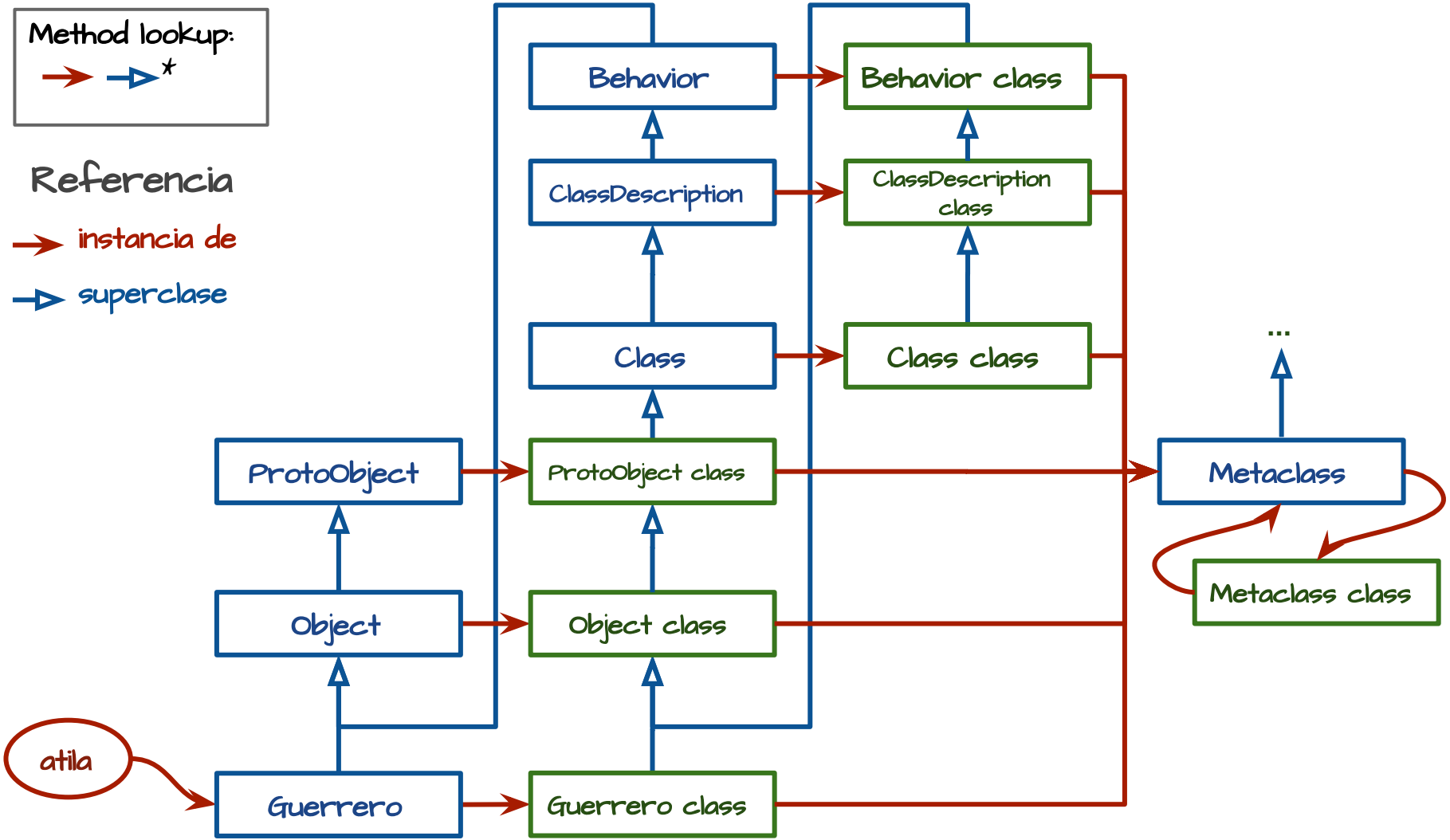
Smalltalk-80

=> Squeak

=> Cuis-Smalltalk

Modelo de ejecución

Estructura del metamodelo


Method lookup:
*
Referencia
instancia de
superclase
Behavior
Behavior class
ClassDescription
ClassDescription class
Class
Class class
...
ProtoObject
ProtoObject class
Metaclass
Metaclass class
Object
Object class
atila
Guerrero
Guerrero class

Method lookup:

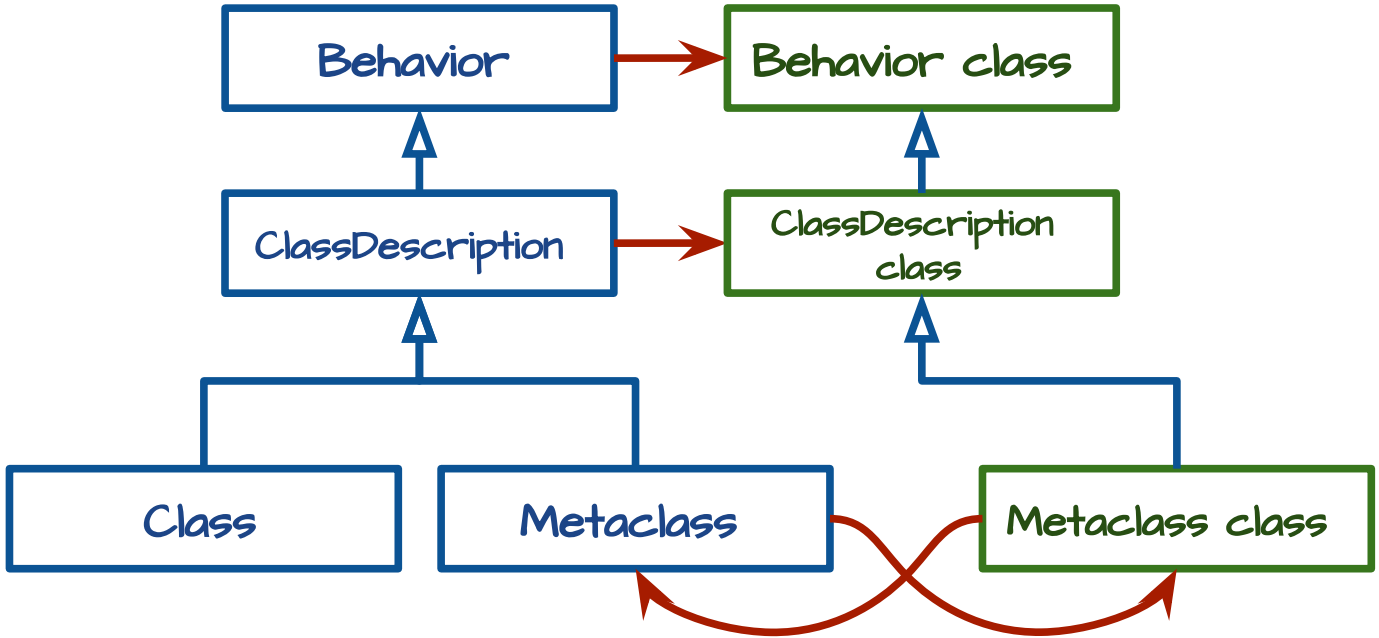
→ →*

Detalle de la jerarquía de Metaclass

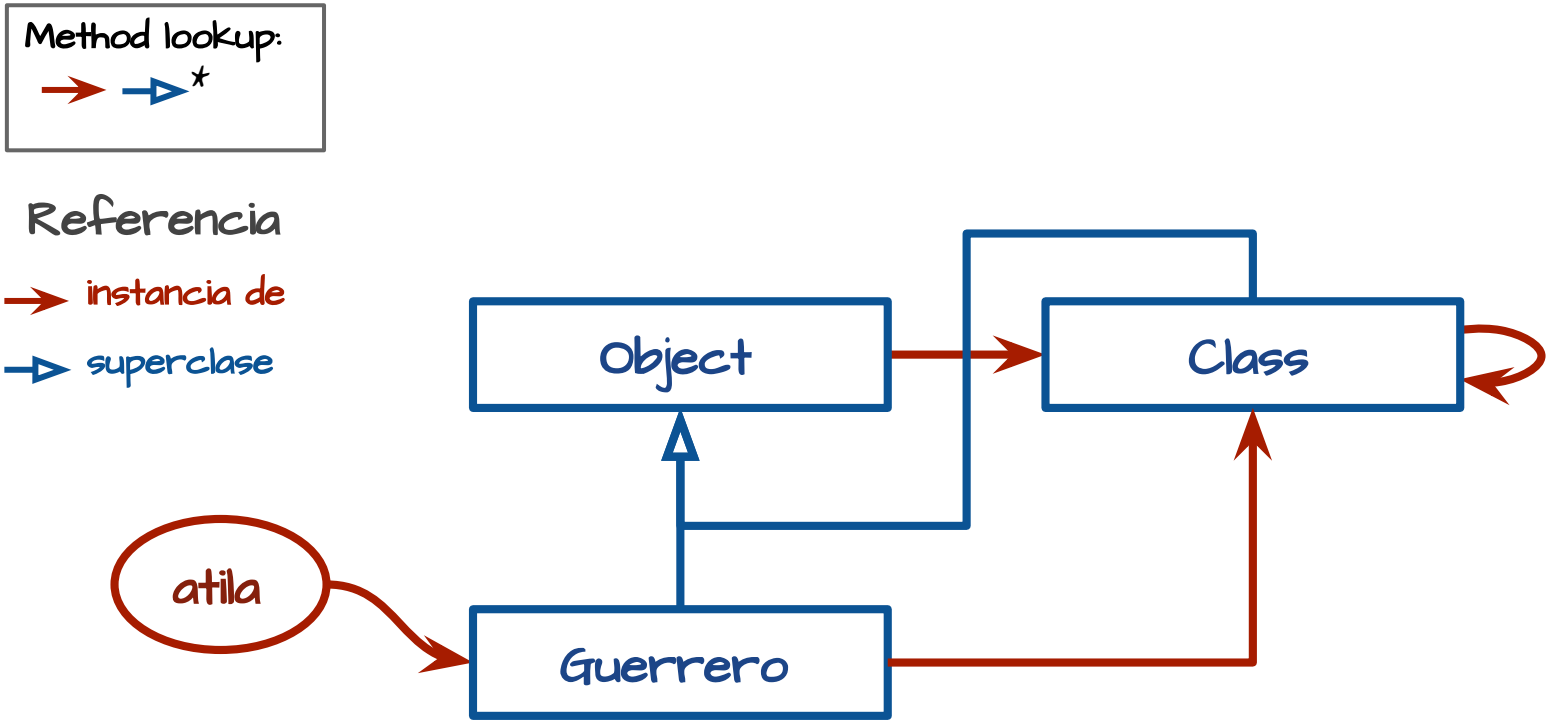
Referencia

→ instancia de

→ superclase



Java


Method lookup:
*
Referencia
instancia de
superclase
Object
Class
atila
Guerrero